3 Lambda to SNS Customized Notifications

Lab 3: Lambda to SNS Customized Notifications

This lab extends Lab 1 and 2 by integrating Lambda to send customized success/failure notifications to SNS after processing an S3 file. This allows for more meaningful alerts compared to raw S3 event notifications.

1. Objective

Modify a Lambda function to send custom success or failure messages to an SNS topic based on the outcome of processing an S3 object.

2. Prerequisites

- Completed **Lab 1: S3 to Lambda Read CSV with Pandas** (Lambda `S3toLambdaRead` with Pandas layer).
- Completed **Lab 2: S3 to SNS Automated Notification** (SNS topic `first_SNS_topic` with confirmed email subscription).
- **Estimated Time:** 25 minutes

3. Detailed Steps

Step 1: Prepare Existing Resources

- **Delete existing Lambda trigger:** In your `S3toLambdaRead` Lambda function, go to **Configuration** → **Triggers**. Select the S3 trigger from Lab 1 and click **Delete**.
- **Delete existing S3 event notification:** In your `bucket-for-sns-lab` S3 bucket, go to **Properties** → **Event notifications**. Select the `SNS-Notification` created in Lab 2 and click **Delete**.

Step 2: Add New S3 Trigger to Lambda

- In your `S3toLambdaRead` Lambda function, click **Add trigger**.
- Select **S3** as the source.
- Choose your S3 bucket: `bucket-for-sns-lab` (from Lab 2).
- Event types: Select **All object create events**.
- Click **Add**.

Step 3: Update Lambda Execution Role Permissions for SNS

Option A: Using AWS Managed Policy (Quick Setup)

- In the Lambda function's **Configuration** tab, go to **Permissions**.
- Click on the **Role name** (e.g., `S3toLambdaRead-role-...`).
- In the IAM role's permissions, click **Add permissions** → **Attach policies**.
- Search for and attach the `AmazonSNSFullAccess` policy.
- Click **Add permissions**.

Option B: Custom Policy (Security Best Practice)

- Follow the same steps above, but instead of attaching a managed policy:
- Click **Add permissions** → **Create inline policy**.
- Use the JSON editor and add:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "sns:Publish"
            ],
            "Resource": "arn:aws:sns:your-region:your-account-id:first_SNS_topic"
        }
    ]
}
```

- Replace `your-region` and `your-account-id` with your actual values.
- Name the policy `SNSPublishPolicy` and click **Create policy**.

Step 4: Modify Lambda Code for SNS Publishing

- In the Lambda function's **Code** tab, update the `lambda_function.py` code.
- **Important:** Copy your SNS topic ARN from the SNS console (e.g., `arn:aws:sns:us-east-1:123456789012:first_SNS_topic`) and update the `SNS_TOPIC_ARN` variable in the code below.

4. Corrected Code (Lab 3)

```
import boto3
import pandas as pd
from io import StringIO
import json

# Your SNS Topic ARN - REPLACE WITH YOUR ACTUAL ARN
SNS_TOPIC_ARN = 'arn:aws:sns:your-region:your-account-id:first_SNS_topic'

def lambda_handler(event, context):
    print("Event received:", event)

    # Initialize S3 and SNS clients
    s3_client = boto3.client('s3')
    sns_client = boto3.client('sns')

    bucket_name = None
    s3_key = None

    try:
        # Parse bucket name and key from the S3 event
        bucket_name = event['Records'][0]['s3']['bucket']['name']  # Note: [0] to
access first record
        s3_key = event['Records'][0]['s3']['object']['key']        # Note: [0] to
access first record

        print(f"Processing file - Bucket: {bucket_name}, Key: {s3_key}")

        # Get the object content from S3
```

```
        response = s3_client.get_object(Bucket=bucket_name, Key=s3_key)
        body = response['Body'].read().decode('utf-8')

        # Read the content into a Pandas DataFrame
        df = pd.read_csv(StringIO(body))
        print(f"DataFrame shape: {df.shape}")
        print("DataFrame head:")
        print(df.head())

        # Publish success message to SNS
        success_message = f'''
File Processing SUCCESS

Bucket: {bucket_name}
File: {s3_key}
Rows processed: {len(df)}
Columns: {list(df.columns)}

Processing completed successfully at {pd.Timestamp.now()}.
        '''.strip()

        sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject='✅ SUCCESS: S3 File Processed',
            Message=success_message,
            MessageStructure='string'
        )
        print("Success notification sent to SNS.")

        return {
            'statusCode': 200,
            'body': json.dumps('Successfully processed S3 file and sent success
notification.')
        }

    except Exception as e:
        print(f"Error processing S3 file: {str(e)}")

        # Improved error handling for SNS notification
        error_bucket = bucket_name if bucket_name else "unknown"
        error_key = s3_key if s3_key else "unknown"

        # Publish failure message to SNS
        failure_message = f'''
File Processing FAILED

Bucket: {error_bucket}
File: {error_key}
Error: {str(e)}

Please check the Lambda function logs for more details.
Timestamp: {pd.Timestamp.now()}
```

```
        '''.strip()

        sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject='❌ FAILED: S3 File Processing Error',
            Subject='❌ FAILED: S3 File Processing Error',
            Message=failure_message,
            MessageStructure='string'
        )
        print("Failure notification sent to SNS.")

        return {
            'statusCode': 500,
            'body': json.dumps(f'Error processing S3 file: {str(e)}')
        }
```

Important: Click **Deploy** after updating the code.

5. Verification (Lab 3)

Test Success Scenario:

1. **Upload a Valid CSV File:**
   - Navigate to your S3 bucket (`bucket-for-sns-lab`).
   - **Delete** any previously uploaded `storedata.csv` to ensure a fresh trigger.
   - Click **Upload** → **Add files**.
   - Select your `storedata.csv` file and click **Upload**.
2. **Check Email:**
   - Open your email inbox.
   - You should receive an email with the subject "✅ **SUCCESS: S3 File Processed**".
   - The message body will contain detailed processing information including row count and column names.

Test Failure Scenario:

1. **Upload an Invalid File:**
   - Create a text file with invalid CSV content (e.g., `invalid.csv` with random text).
   - Upload this file to trigger a processing error.
2. **Check Email:**
   - You should receive an email with the subject "❌ **FAILED: S3 File Processing Error**".
   - The message will contain error details.

Monitor Lambda Logs:

- Go to **CloudWatch** → **Log groups** → `/aws/lambda/S3toLambdaRead` .
- Check the latest log stream to see detailed execution logs.

6. Troubleshooting

Common Issues:

1. **Permission Denied Error:** Ensure the Lambda execution role has SNS publish permissions.
2. **Invalid Topic ARN:** Double-check that your SNS_TOPIC_ARN variable matches your actual SNS topic ARN.
3. **No Email Received:** Verify your email subscription is confirmed in the SNS topic.
4. **Lambda Timeout:** If processing large files, consider increasing the Lambda timeout in Configuration → General configuration.

7. Key Improvements Made

- **Fixed S3 event parsing:** Added `[0]` to access the first record in the Records array.
- **Corrected SNS parameter:** Changed `TargetArn` to `TopicArn`.
- **Enhanced error handling:** Prevents undefined variable errors in exception blocks.
- **Improved notifications:** Added more detailed success/failure messages with timestamps and data insights.
- **Security best practice:** Provided option for least-privilege IAM policy.
- **Better logging:** Enhanced print statements for easier debugging.

8. Cleanup (Lab 3)

1. **Delete Lambda Function:**
   - In the Lambda console, select `S3toLambdaRead` and click **Actions** → **Delete function**. This will also delete its S3 trigger.
2. **Delete S3 Bucket:**
   - If you created `bucket-for-sns-lab` solely for this lab, empty it first then delete it.
3. **Delete SNS Topic and Subscription:**
   - If you created `first_SNS_topic` and its subscription solely for this lab, delete them from the SNS console.
4. **Clean up IAM Role (Optional):**
   - If you added SNS permissions to the Lambda role, you may remove them if no longer needed.

Next Steps: Consider exploring Lambda error handling patterns, SNS message formatting, or integrating with other AWS services like DynamoDB for data storage.